



TECNOLÓGICO
NACIONAL DE MÉXICO®



ACTIVIDAD 1

PROGRAMACIÓN DEL LADO DEL CLIENTE VS DEL LADO DEL SERVIDOR

La arquitectura web moderna funciona mediante el **modelo cliente-servidor**, un patrón de distribución de tareas que divide el trabajo entre los proveedores de un recurso o servicio (servidores) y los demandantes de dicho servicio (clientes).

1. Programación del lado del cliente (*Client-Side*)

La programación *client-side* engloba todo el código y la lógica que se ejecutan directamente en la aplicación cliente, casi siempre un navegador web (como Chrome, Firefox, Safari o Edge).

Funciones principales

- **Presentación y Estructura:** Renderizar la interfaz visual con la que interactúa el usuario final.
- **Manipulación del DOM (Document Object Model):** Modificar el contenido, estilo o estructura de la página en tiempo real según la interacción del usuario sin necesidad de recargar la página completa.
- **Validaciones de Entrada:** Comprobar que los formularios contengan formatos correctos (por ejemplo, correo electrónico válido) antes de enviar la información al servidor.
- **Gestión del Estado de la Interfaz:** Controlar menús desplegados, modales, temas oscuros/claros, animaciones e intenciones del usuario en tiempo real.

Cómo funciona

1. El usuario ingresa una URL en el navegador.
2. El servidor responde enviando archivos estáticos: documentos **HTML** (estructura), hojas de estilo **CSS** (diseño) y scripts de **JavaScript** (comportamiento).
3. El procesador del dispositivo del usuario (CPU/GPU) ejecuta el código JS para construir la experiencia interactiva.

2. Programación del lado del servidor (*Server-Side*)

La programación *server-side* abarca la lógica que se ejecuta de forma remota en los servidores donde reside la aplicación. Es la encargada de procesar las solicitudes entrantes, coordinar las reglas de negocio y comunicarse con las bases de datos.

Funciones principales

- **Procesamiento de Reglas de Negocio:** Calcular pagos, procesar compras, autorizar transacciones y manipular lógica compleja de la aplicación.
- **Autenticación y Autorización:** Verificar credenciales (usuarios y contraseñas), gestionar sesiones, tokens (JWT) y controlar qué información puede ver cada usuario.
- **Gestión de Bases de Datos:** Consultar (CRUD: crear, leer, actualizar, borrar) datos guardados en sistemas como MySQL, PostgreSQL o MongoDB.
- **Seguridad y Privacidad:** Ocultar claves secretas, APIs internas y algoritmos sensibles para evitar que sean expuestos al público.
- **Generación Dinámica de Contenido:** Preparar las respuestas personalizadas que se enviarán de vuelta al cliente.

Cómo funciona

1. El navegador del cliente envía una petición HTTP (por ejemplo, POST /login con un usuario y contraseña).
2. El entorno del servidor (ej. Node.js, Python/Django, PHP) recibe la petición y la procesa.
3. El servidor se conecta a la base de datos para validar las credenciales.
4. Genera una respuesta (por lo general en formato **JSON** para APIs o un HTML renderizado) y se la envía de regreso al cliente.

CUADRO COMPARATIVO

Aspecto	Programación del lado del cliente (Client-Side)	Programación del lado del servidor (Server-Side)
Descripción general	Gestiona la interfaz con la que interactúa el usuario y la presentación visual de la aplicación.	Gestiona la lógica interna, el procesamiento de datos y la conexión con bases de datos.
Lenguajes más utilizados	JavaScript, TypeScript, HTML y CSS.	JavaScript (Node.js), Python, PHP, Java, C#, Ruby, Go.
Tecnologías y herramientas comunes	React, Angular, Vue.js, Svelte, Tailwind CSS, Bootstrap, Vite.	Express.js, Django, Laravel, Spring Boot, .NET, PostgreSQL, MongoDB, MySQL.
Ventajas	• Respuesta visual rápida e interactiva. • Reduce la carga de trabajo en el servidor • Mejora la experiencia de usuario (UX).	• Mayor seguridad en datos sensibles. • Capacidad para manejar lógica compleja y bases de datos. • Compatibilidad universal independiente del hardware del cliente.
Desventajas	• Depende del rendimiento e interpretación del navegador del cliente. • Código expuesto y visible para cualquier usuario. • Menor capacidad para procesar grandes volúmenes de datos.	• Incrementa el consumo de recursos en el servidor. • Cada solicitud puede aumentar la latencia si la red es lenta. • Mayor costo de infraestructura.
Seguridad	Menor nivel de seguridad; el código fuente es público y vulnerable a manipulaciones directas (como ataques XSS).	Mayor nivel de seguridad; el código permanece privado dentro del servidor, protegiendo credenciales y datos confidenciales.
Ubicación de ejecución	Navegador web del dispositivo del usuario.	Servidor web (físico o en la nube).